Python Garbage Collection

Research on automatic memory reclamation and hands-on experiments with the `gc` module

Interpreter used	CPython 3.12.3 (Linux)
Scope	Reference counting, cyclic GC, generations, thresholds, manual control, debugging, performance
Contents	Research notes, gc API reference, 7 runnable experiments (code + real captured output), best practices
Date	3 November 2026

Contents

Part	Topic
1	Research: how Python manages memory (reference counting + cyclic garbage collector)
2	Research: generations, thresholds and the collection algorithm
3	Research: version history and what changed recently
4	The gc module: API quick reference
5	Experiments 1-7 (code, output, analysis)
6	Best practices and common pitfalls
7	Summary, how to re-run, and sources

Note on the experiments. Every code listing in Part 5 is the exact file that was executed, and every output block is the real, unedited output of that run. Numbers such as object counts and timings depend on the Python version and machine, so your results will be close but not identical.

1. How Python Manages Memory

CPython (the reference implementation of Python) uses **two cooperating mechanisms** to reclaim memory automatically. Programmers rarely need to think about either, but understanding them explains puzzling behaviour such as memory that is not released, destructors running at odd times, or pauses in a latency-sensitive program.

1.1 Mechanism one: reference counting

Every Python object carries a counter, `ob_refcnt`, which records how many references point to it. The counter goes up when you bind a name, append to a container, or pass the object around, and goes down when a name is deleted, rebinds, or goes out of scope.

- When the count reaches **0**, the object is destroyed *immediately*: its `__del__` runs, its memory returns to the allocator, and the objects it referenced have their counts decremented in turn (a cascade).
- This is deterministic and cheap: memory is freed at the exact moment the last reference disappears. This is why `del x` on a file or lock wrapper often releases the resource right away.
- Weak references (`weakref`) do not increment the count, so they can observe an object without keeping it alive.
- Drawback: the counter must be updated on almost every operation (costly under multithreading), and it **cannot free reference cycles**.

1.2 Mechanism two: the cyclic garbage collector

A reference cycle exists when objects refer to each other directly or indirectly, e.g. A refers to B and B refers to A, or a list that contains itself. Even after the program drops every outside reference, each object in the cycle still has a count of at least 1, so reference counting alone never frees them.

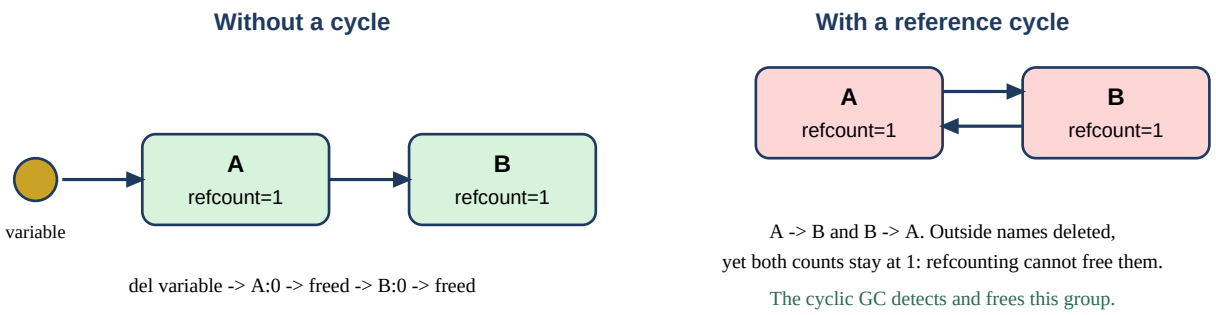

Figure 1: reference counting frees acyclic objects instantly but cannot free a cycle.

The `gc` module exposes the **cyclic garbage collector**, which supplements reference counting. It only examines objects that can take part in cycles, i.e. *container* objects (lists, user-defined instances, most dicts, sets, tuples that hold containers, etc.). Atomic objects such as `int`, `str` and `float` are never tracked (Experiment 4 verifies this).

Because the collector only supplements reference counting, the official documentation notes that you can disable it if you are sure your program creates no reference cycles.

1.3 How the collector finds garbage

The algorithm works on the set of tracked container objects it is examining (a trial-deletion approach):

- **Copy counts.** For each object in the examined generation, copy its refcount into a temporary field (`gc_refs`).
- **Subtract internal references.** Traverse each object's referents (using its `tp_traverse` slot); for every reference *between examined objects*, decrement the referent's `gc_refs`. After this, any object with `gc_refs > 0` is being referenced from *outside* the examined set, so it is definitely reachable.
- **Mark reachable.** Everything reachable from those externally referenced objects is also alive.
- **Sweep.** Whatever remains is unreachable cyclic garbage. The collector runs finalizers (`__del__`, weakref callbacks), clears the objects' references to break the cycles, and lets reference counting free the memory.

2. Generations, Thresholds and Triggering

2.1 Generational hypothesis

Scanning every object on every pass would be slow. The collector exploits the **generational hypothesis**: most objects die young, so recently created objects are the best candidates. Objects are grouped into three generations by how many collections they have survived.

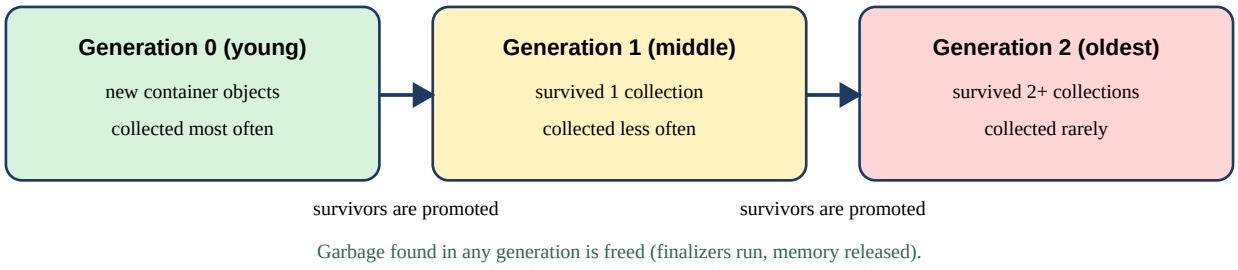

Figure 2: three-generation design. Survivors are promoted to the next older generation.

2.2 What triggers an automatic collection?

Collection is driven by **allocation counts**, not by a timer or by memory pressure:

- The interpreter tracks (container allocations minus deallocations) since the last collection. When this exceeds **threshold0**, a collection of the young generation starts.
- If generation 0 has been examined more than **threshold1** times since generation 1 was last examined, generation 1 is examined too (and so on for threshold2).
- Setting `threshold0` to **0** disables automatic collection (equivalent in effect to `gc.disable()`).
- For the oldest generation there is an extra rule: a full collection is skipped unless the number of long-lived objects that have not been examined yet is large enough relative to the total, so that cost stays proportional (documentation: "With the third generation, things are a bit more complicated").

The defaults seen in our run are shown by `gc.get_threshold()`; Experiment 3 prints them and then varies threshold0 to show how the number of automatic collections changes.

2.3 Objects that cannot be collected: the history of `__del__`

Before Python 3.4, a cycle containing an object with a `__del__` method could not be collected safely and was parked in `gc.garbage`. **PEP 442** (Python 3.4) fixed this: such objects are now finalized and collected normally. Today `gc.garbage` should be empty most of the time, except for certain C extension types. Experiment 5 demonstrates this.

3. Version History: What Changed Recently

Version	Change
3.2	A ResourceWarning is emitted at shutdown if gc.garbage is non-empty.
3.4	PEP 442: objects with __del__ in cycles are collectable; gc.get_stats() added (gc.callbacks arrived in 3.3).
3.7	gc.freeze(), gc.unfreeze(), gc.get_freeze_count() added (help copy-on-write when forking).
3.8	gc.get_objects() gained the generation parameter.

Version	Change
3.9	gc.is_finalized() added.
3.13 / 3.14	Work on an **incremental** collector. In 3.14 the middle generation was removed and threshold2 ignored. The 3.14.5 documentation notes generation 1 was **reintroduced** and threshold2 restored so GC behaviour matches 3.13.
3.14	gc.collect(1) briefly meant "perform an increment of collection"; in 3.14.5 it again collects the middle generation.
Free-threaded build	Also checks growth in process memory before running the collector.
3.15	gc.get_stats() dictionaries gain duration and candidates keys; callback info gains the same.

Practical takeaway: the public API (`collect`, `get_threshold`, `set_threshold`, `callbacks`) is stable, but the exact default thresholds and internal scheduling differ between versions. Always query `gc.get_threshold()` on your own interpreter rather than assuming 700/10/10.

4. The gc Module: API Quick Reference

Function / attribute	Purpose
gc.enable() / gc.disable() / gc.isenabled()	Turn automatic collection on or off, or query its state. Manual gc.collect() still works while disabled.
gc.collect(generation=2)	Run a collection now. Returns the number of unreachable objects found (collected + uncollectable).
gc.get_threshold() / gc.set_threshold(t0[, t1[, t2]])	Read or change collection frequency. t0 = 0 disables automatic collection.
gc.get_count()	Tuple of current counters (count0, count1, count2) used for triggering.
gc.get_stats()	List of three dicts (per generation): collections, collected, uncollectable.
gc.get_objects(generation=None)	All objects tracked by the collector (optionally only one generation).
gc.get_referrers(*objs) / gc.get_referents(*objs)	Who points to these objects / what do they point to. Debugging aid only.
gc.is_tracked(obj) / gc.is_finalized(obj)	Is the object tracked by the GC / has it been finalized by the GC.
gc.set_debug(flags) / gc.get_debug()	Flags: DEBUG_STATS, DEBUG_COLLECTABLE, DEBUG_UNCOLLECTABLE, DEBUG_SAVEALL, DEBUG_LEAK. Output goes to stderr.
gc.garbage	List of unreachable objects the collector found but could not free (or all garbage when DEBUG_SAVEALL is set).
gc.callbacks	List of callables invoked as f(phase, info) at the start and stop of every collection (phase = 'start' or 'stop'; info has generation, collected, uncollectable).
gc.freeze() / gc.unfreeze() / gc.get_freeze_count()	Move all current objects to a permanent generation that is ignored by future collections (or move them back).

Warning: the effect of calling `gc.collect()` while a collection is already in progress (for example from inside a callback) is undefined. Do not do it.

5. Experiments

Each experiment states an objective, shows the complete source, the real output, and an analysis. All experiments are standalone: run them with `python expN_name.py`.

Experiment 1: Reference counting in action

Objective. Show how reference counts rise and fall and that an object is destroyed the instant its count hits zero, with no help from the cyclic GC.

INPUT: exp1_refcount.py

```
"""Experiment 1: Reference counting - CPython's primary memory manager."""
import sys
import weakref


class Resource:
    def __init__(self, name):
        self.name = name

    def __del__(self):
        print(f"  [__del__] {self.name} destroyed")


print("1) Reference count of an object")
a = Resource("A")
print("  refcount after creation :", sys.getrefcount(a) - 1)  # -1: the argument itself
b = a
print("  refcount after b = a    :", sys.getrefcount(a) - 1)
lst = [a, a]
print("  refcount after list x2  :", sys.getrefcount(a) - 1)

print("\n2) Decrementing references")
del lst
print("  after del lst           :", sys.getrefcount(a) - 1)
del b
print("  after del b             :", sys.getrefcount(a) - 1)

print("\n3) Last reference removed -> immediate destruction (no GC needed)")
ref = weakref.ref(a)            # weak refs do NOT increase the refcount
print("  alive before del a      :", ref() is not None)
del a                           # refcount hits 0 -> __del__ runs right here
print("  alive after del a       :", ref() is not None)

print("\n4) Refcount of small ints / interned strings is shared")
x = 256
print("  refcount(256) is large (shared singleton):", sys.getrefcount(x) > 5)
```

OUTPUT (captured from a real run)

```
1) Reference count of an object
  refcount after creation : 1
  refcount after b = a    : 2
  refcount after list x2  : 4

2) Decrementing references
  after del lst           : 2
  after del b             : 1

3) Last reference removed -> immediate destruction (no GC needed)
  alive before del a      : True
  [__del__] A destroyed
  alive after del a       : False

4) Refcount of small ints / interned strings is shared
  refcount(256) is large (shared singleton): True
```

Analysis

- `sys.getrefcount()` always reports one extra reference (its own argument), hence the `- 1`.
- Creating `b = a` and storing a twice in a list raised the count from 1 to 4; deleting them brought it back to 1.
- `__del__` fired exactly at `del a`, before the next print. That is deterministic reference-count destruction.
- The weak reference did not change the count and correctly became dead after the object vanished.

Experiment 2: Reference cycles and manual collection

Objective. Prove that cycles leak when the collector is disabled and that `gc.collect()` reclaims them.

INPUT: exp2_cycles.py

```
"""Experiment 2: Reference cycles - what reference counting cannot free."""
import gc
import weakref


class Node:
    def __init__(self, name):
        self.name = name
        self.other = None


def make_cycle():
    a, b = Node("a"), Node("b")
    a.other = b
    b.other = a                  # a -> b -> a  (cycle)
    return weakref.ref(a), weakref.ref(b)


gc.collect()                     # start from a clean slate
gc.disable()                     # turn OFF automatic collection
print("Automatic GC enabled?:", gc.isenabled())

ra, rb = make_cycle()            # local names a, b are gone after return
print("\nAfter function returns (no outside references remain):")
print("  node a alive:", ra() is not None)
print("  node b alive:", rb() is not None, " <-- leaked by refcounting alone")

n = gc.collect()                 # manual collection
print(f"\ngc.collect() returned {n}  (unreachable objects found)")
print("  node a alive:", ra() is not None)
print("  node b alive:", rb() is not None)

# Control: acyclic objects are freed instantly even with GC disabled
c = Node("c")
rc = weakref.ref(c)
del c
print("\nControl (no cycle), GC still disabled:")
print("  node c alive after del:", rc() is not None)

# Self-referencing list
l = []
l.append(l)
del l
print("\nSelf-referencing list collected by gc.collect():", gc.collect() >= 1)
gc.enable()
print("Automatic GC re-enabled:", gc.isenabled())
```

OUTPUT (captured from a real run)

```
Automatic GC enabled?: False

After function returns (no outside references remain):
  node a alive: True
  node b alive: True  <-- leaked by refcounting alone

gc.collect() returned 2  (unreachable objects found)
  node a alive: False
  node b alive: False

Control (no cycle), GC still disabled:
  node c alive after del: False

Self-referencing list collected by gc.collect(): True
Automatic GC re-enabled: True
```

Analysis

- With automatic GC disabled, both nodes stayed alive even though nothing outside referred to them: refcounting alone could not free the cycle.
- `gc.collect()` returned 2 on Python 3.12: the two Node objects (older versions may also count their instance dicts), and both weak references died.
- The control object *without* a cycle was freed immediately even with GC disabled, confirming the two mechanisms are independent.
- A list that contains itself is also a cycle and was collected manually.

Experiment 3: Thresholds, counts and automatic triggering

Objective. Observe how `threshold0` controls the frequency of automatic collections, using `gc.callbacks` to log them.

INPUT: exp3_thresholds.py

```
"""Experiment 3: Thresholds, counts and automatic triggering."""
import gc

print("Default thresholds     :", gc.get_threshold())
print("Current counts (g0,g1,g2):", gc.get_count())

# Log every automatic collection using gc.callbacks
events = []


def tracker(phase, info):
    if phase == "stop":
        events.append((info["generation"], info["collected"]))


gc.callbacks.append(tracker)
gc.collect()
events.clear()


class Box:
    pass


def churn(n):
    """Create n small reference cycles (each Box points to itself)."""
    for _ in range(n):
        b = Box()
        b.me = b


for limit in (700, 100, 5000):
    gc.set_threshold(limit, 10, 10)
    events.clear()
    churn(20000)
    gen0 = sum(1 for g, _ in events if g == 0)
    total = sum(c for _, c in events)
    print(f"\nthreshold0={limit:<5}-> automatic collections: {len(events):>4}"
          f"  (gen0: {gen0}), objects freed: {total}")

print("\nSetting threshold0 = 0 disables automatic collection")
gc.set_threshold(0)
events.clear()
churn(20000)
print("  automatic collections:", len(events))
print("  garbage waiting for us:", gc.collect(), "objects freed by manual gc.collect()")

gc.callbacks.remove(tracker)
gc.set_threshold(700, 10, 10)
```

OUTPUT (captured from a real run)

```
Default thresholds     : (700, 10, 10)
Current counts (g0,g1,g2): (199, 5, 0)

threshold0=700  -> automatic collections:   28  (gen0: 26), objects freed: 19577

threshold0=100  -> automatic collections:  200  (gen0: 183), objects freed: 20314

threshold0=5000 -> automatic collections:    4  (gen0: 4), objects freed: 19995

Setting threshold0 = 0 disables automatic collection
  automatic collections: 0
  garbage waiting for us: 20114 objects freed by manual gc.collect()
```

Analysis

- With threshold0 = 100 the GC ran about 200 times; with 700 about 28 times; with 5000 only 4 times. Lower threshold means more frequent (smaller) collections.
- In all three cases roughly the same ~20,000 cyclic objects were freed: the threshold changes *when* garbage is freed, not *whether*.
- With threshold0 = 0, no automatic collection ran. All 20,000+ garbage objects piled up and were only freed by our manual `gc.collect()`.
- The initial `gc.get_count()` value varies by run, since it depends on what the interpreter allocated during start-up.

Experiment 4: Generations, promotion and tracked objects

Objective. Watch survivors move between generations, read `gc.get_stats()`, and learn which objects the GC tracks.

INPUT: exp4_generations.py

```
"""Experiment 4: Generations, promotion of survivors and gc.get_stats()."""
import gc


class Item:
    pass


gc.collect()
gc.disable()

def gen_sizes():
    return [len(gc.get_objects(generation=g)) for g in (0, 1, 2)]

print("Objects per generation (start)     :", gen_sizes())

keep = [Item() for _ in range(1000)]          # survivors
print("After creating 1000 Items          :", gen_sizes())

gc.collect(0)
print("After gc.collect(0)  (young only)  :", gen_sizes())

gc.collect(1)
print("After gc.collect(1)                :", gen_sizes())

gc.collect(2)
print("After gc.collect(2)  (full)        :", gen_sizes())
print("\nSurvivors are promoted: young -> middle -> oldest.")

gc.enable()
print("\nPer-generation statistics (since interpreter start):")
for i, s in enumerate(gc.get_stats()):
    print(f"  gen{i}: collections={s['collections']:<4} "
          f"collected={s['collected']:<6} uncollectable={s['uncollectable']}")

print("\nWhich objects does the GC even track?")
for label, obj in [("int 5", 5), ("str 'a'", "a"), ("tuple of ints (1,2)", (1, 2)),
                   ("list []", []), ("dict {}", {}), ("dict {'a':1}", {"a": 1}),
                   ("dict {'a':[]}", {"a": []}),
                   ("Item() instance", Item())]:
    print(f"  {label:<22} tracked = {gc.is_tracked(obj)}")
```

OUTPUT (captured from a real run)

```
Objects per generation (start)     : [3, 0, 5058]
After creating 1000 Items          : [1006, 0, 5058]
After gc.collect(0)  (young only)  : [2, 1004, 5058]
After gc.collect(1)                : [2, 0, 6062]
After gc.collect(2)  (full)        : [2, 0, 6057]

Survivors are promoted: young -> middle -> oldest.

Per-generation statistics (since interpreter start):
  gen0: collections=6    collected=24     uncollectable=0
  gen1: collections=1    collected=0      uncollectable=0
  gen2: collections=2    collected=0      uncollectable=0

Which objects does the GC even track?
  int 5                  tracked = False
  str 'a'                tracked = False
  tuple of ints (1,2)    tracked = False
  list []                tracked = True
  dict {}                tracked = False
  dict {'a':1}           tracked = False
  dict {'a':[]}          tracked = True
  Item() instance        tracked = True
```

Analysis

- 1,000 new `Item` objects appeared in generation 0 (the count rose by 1,003: the 1,000 items, the list holding them, and a few interpreter objects).
- `gc.collect(0)` moved those survivors into generation 1; `gc.collect(1)` moved them into generation 2 (the oldest). Survivors are never freed because the list `keep` still references them.
- Integers, strings and tuples of atomic values are not tracked. An empty dict and a dict of atomic values are also untracked, but `{'a': []}` is tracked because it contains a container. This is a CPython optimisation that reduces collector work.

Experiment 5: Debugging tools

Objective. Use `get_referents`, `get_referrers`, `DEBUG_SAVEALL` and `gc.garbage`; confirm that `__del__` in a cycle is handled (PEP 442).

INPUT: exp5_debugging.py

```
"""Experiment 5: Debugging - get_referrers/referents, DEBUG_SAVEALL, gc.garbage."""
import gc


class Parent:
    def __init__(self):
        self.child = Child(self)


class Child:
    def __init__(self, parent):
        self.parent = parent


gc.collect()

print("A) get_referents / get_referrers")
p = Parent()
print("  Referents of the Parent instance's __dict__:",
      [type(o).__name__ for o in gc.get_referents(p.__dict__)])
print("  Who refers to the Child?:",
      sorted({type(o).__name__ for o in gc.get_referrers(p.child)}))

print("\nB) DEBUG_SAVEALL keeps garbage inspectable in gc.garbage")
del p
gc.set_debug(gc.DEBUG_SAVEALL)
found = gc.collect()
gc.set_debug(0)
print("  gc.collect() found:", found, "objects")
print("  gc.garbage contains:", sorted(type(o).__name__ for o in gc.garbage))
print("  (objects are kept alive in gc.garbage until we clear it)")
gc.garbage.clear()
print("  After gc.garbage.clear(), another collect frees:", gc.collect())

print("\nC) __del__ inside a cycle (safe since Python 3.4, PEP 442)")
class Loud:
    def __init__(self):
        self.me = self
    def __del__(self):
        print("  Loud.__del__ called by the GC")
Loud();
n = gc.collect()
print("  collected:", n, "| gc.garbage empty:", gc.garbage == [])

print("\nD) gc.set_debug(gc.DEBUG_STATS) output goes to stderr:")
print("  flags available:", [f for f in dir(gc) if f.startswith("DEBUG_")])
```

OUTPUT (captured from a real run)

```
A) get_referents / get_referrers
  Referents of the Parent instance's __dict__: ['Child']
  Who refers to the Child?: ['dict']

B) DEBUG_SAVEALL keeps garbage inspectable in gc.garbage
  gc.collect() found: 3 objects
  gc.garbage contains: ['Child', 'Parent', 'dict']
  (objects are kept alive in gc.garbage until we clear it)
  After gc.garbage.clear(), another collect frees: 3

C) __del__ inside a cycle (safe since Python 3.4, PEP 442)
  Loud.__del__ called by the GC
  collected: 1 | gc.garbage empty: True

D) gc.set_debug(gc.DEBUG_STATS) output goes to stderr:
  flags available: ['DEBUG_COLLECTABLE', 'DEBUG_LEAK', 'DEBUG_SAVEALL', 'DEBUG_STATS', 'DEBUG_UNCOLLECTABLE']
```

Analysis

- The Parent/Child cycle was visible through the referents and referrers APIs.
- With `DEBUG_SAVEALL`, garbage is *not* freed; it is saved in `gc.garbage` so you can inspect it. Three objects were found: Parent, Child and one dict. Clearing the list lets a later collection free them.
- The `Loud` object with `__del__` in a self-cycle was finalized and collected, and `gc.garbage` stayed empty. In Python 2 and early Python 3 it would have leaked.
- `DEBUG_STATS` and friends print to stderr and are best used interactively.

Experiment 6: The cost of the GC, disable and freeze

Objective. Measure how the collector affects allocation-heavy code and demonstrate `gc.freeze()`.

INPUT: exp6_performance.py

```
"""Experiment 6: Cost of the GC, gc.disable(), and gc.freeze()."""
import gc
import time


def build(n):
    """Allocate many long-lived container objects (acyclic)."""
    return [[i] for i in range(n)]


def timed(label, n=2_000_000):
    t = time.perf_counter()
    data = build(n)
    dt = time.perf_counter() - t
    print(f"  {label:<28}{dt:6.3f} s")
    return dt

print("Allocating 2,000,000 small lists (timings vary by machine):")
gc.enable();  gc.collect()
t_on = timed("GC enabled (default)")
gc.disable(); gc.collect()
t_off = timed("GC disabled")
gc.enable()
print(f"  -> speed-up from disabling GC: {t_on / t_off:.2f}x")

print("\ngc.freeze(): move current objects to a permanent generation")
big = build(300_000)
before = gc.get_freeze_count()
gc.freeze()
print("  frozen objects before / after:", before, "/", gc.get_freeze_count())
t = time.perf_counter(); gc.collect(); dt1 = time.perf_counter() - t
gc.unfreeze()
t = time.perf_counter(); gc.collect(); dt2 = time.perf_counter() - t
print(f"  full collect with frozen heap  : {dt1*1000:8.2f} ms")
print(f"  full collect after unfreeze    : {dt2*1000:8.2f} ms")
```

OUTPUT (captured from a real run)

```
Allocating 2,000,000 small lists (timings vary by machine):
  GC enabled (default)         1.182 s
  GC disabled                  0.254 s
  -> speed-up from disabling GC: 4.65x

gc.freeze(): move current objects to a permanent generation
  frozen objects before / after: 375 / 305425
  full collect with frozen heap  :     0.01 ms
  full collect after unfreeze    :    23.60 ms
```

Analysis

- Allocating 2 million small lists was several times faster with the GC disabled, because each young-generation collection repeatedly scans the growing set of survivors. (Timings are machine dependent.)
- `gc.freeze()` moved ~305,000 objects into the permanent generation. A full collection with that heap frozen took almost no time, compared to tens of milliseconds after `unfreeze()`.
- Real-world use: disable the GC while building a big data structure, then re-enable; or freeze after start-up in a pre-fork server so child processes do not dirty shared memory pages.

- **Caution:** disabling the GC is only safe for code that does not create cycles, or when you collect manually at a safe point.

Experiment 7: A realistic leak and its fix

Objective. Show a common leak pattern (an object storing a bound method of itself) and measure memory with `tracemalloc`.

INPUT: exp7_memory_leak.py

```
"""Experiment 7: A real leak pattern - request handlers with cycles - and the fix."""
import gc
import tracemalloc


class Request:
    def __init__(self):
        self.payload = bytearray(100_000)      # ~100 KB
        self.callback = self.done      # bound method -> refers to self (cycle)

    def done(self):
        pass


def handle(n):
    for _ in range(n):
        Request()


gc.collect()
gc.disable()
tracemalloc.start()
handle(200)
cur, _ = tracemalloc.get_traced_memory()
print(f"After 200 requests, GC disabled   : {cur/1e6:7.1f} MB still allocated")
gc.collect()
cur, _ = tracemalloc.get_traced_memory()
print(f"After manual gc.collect()         : {cur/1e6:7.1f} MB still allocated")

class FixedRequest:
    def __init__(self):
        import weakref
        self.payload = bytearray(100_000)
        self.callback = weakref.WeakMethod(self.done)   # weak -> no cycle
    def done(self):
        pass

def handle_fixed(n):
    for _ in range(n):
        FixedRequest()

handle_fixed(200)
cur, _ = tracemalloc.get_traced_memory()
print(f"After 200 FixedRequests (weakref) : {cur/1e6:7.1f} MB  (freed without GC)")
gc.enable()
```

OUTPUT (captured from a real run)

```
After 200 requests, GC disabled   :    20.0 MB still allocated
After manual gc.collect()         :     0.0 MB still allocated
After 200 FixedRequests (weakref) :     0.1 MB  (freed without GC)
```

Analysis

- Each `Request` stores `self.done`, a bound method that references the instance, creating a cycle. With the GC off, 200 requests kept about 20 MB (200 x 100 KB) alive.

- A manual `gc.collect()` released it all.
- Using `weakref.WeakMethod` removes the cycle, so memory is released by plain reference counting with no GC involved. Fixing the design is better than relying on the collector.

6. Best Practices and Pitfalls

- **Do not disable the GC by default.** Cycles are surprisingly easy to create (callbacks, parent/child links, exceptions holding tracebacks that reference frames, caches).
- **Prefer breaking cycles by design:** use `weakref` (`WeakMethod`, `WeakValueDictionary`) for back-references and observers.
- **Use context managers** (`with`) for files, sockets and locks. Do not rely on `__del__` or on the GC to release external resources.
- **Tune, don't guess:** measure with `gc.callbacks` and `gc.get_stats()` before changing thresholds. Raising threshold0 trades memory for fewer pauses.
- **Collect at safe points:** in latency-critical programs (games, trading, request handlers), disable automatic GC and call `gc.collect()` between frames or requests.
- **Pre-fork servers:** `gc.disable()` early in the parent, `gc.freeze()` right before `fork()`, `gc.enable()` in children, to preserve copy-on-write sharing (as recommended by the documentation).
- **Hunt leaks** with `gc.set_debug(gc.DEBUG_LEAK)`, `gc.get_objects()` type counts, and `tracemalloc`.
- **Do not call** `gc.collect()` inside a GC callback or inside `__del__` handlers that may be triggered by a collection.
- Remember that freeing memory in Python does not always return it to the operating system; the allocator may keep arenas for reuse.

Quick decision guide

Situation	Recommended action
Normal application	Leave the GC enabled with default thresholds.
Building a huge acyclic structure at start-up	gc.disable() during build, gc.enable() after (optionally gc.freeze()).
Frequent small pauses hurting latency	Raise threshold0 or disable and collect at safe points.
Memory keeps growing	Run gc.collect(); if memory drops, you have cycles. Inspect with DEBUG_SAVEALL / gc.garbage.
Forking workers	disable -> freeze -> fork -> enable in children.

7. Summary, Re-running and Sources

Summary. CPython frees most objects instantly via reference counting. A supplementary, generational, cyclic garbage collector, controlled through the gc module, handles reference cycles. It triggers by allocation-count thresholds, examines young objects most often, and can be tuned, disabled, invoked manually, frozen, and inspected. Our experiments confirmed each claim with measurable output: cycles leak without the collector (Exp. 2), thresholds drive frequency (Exp. 3), survivors are promoted (Exp. 4), debugging data is available (Exp. 5), the GC costs time in allocation-heavy code (Exp. 6), and weak references remove a common leak (Exp. 7).

How to re-run

```
python3 exp1_refcount.py
python3 exp2_cycles.py
python3 exp3_thresholds.py
python3 exp4_generations.py
python3 exp5_debugging.py
python3 exp6_performance.py
python3 exp7_memory_leak.py
```

Sources

- Python documentation, *gc - Garbage Collector interface*: https://docs.python.org/3/library/gc.html
- Python Developer's Guide, *Garbage collector design*: https://devguide.python.org/internals/garbage-collector/
- PEP 442, *Safe object finalization*: https://peps.python.org/pep-0442/
- Python documentation, *weakref* and *tracemalloc* modules: https://docs.python.org/3/library/